

DevOps. Automatización y monitorización de infraestructuras

Unidad 01. Introducción al curso y entorno de trabajo



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 01. Introducción al curso y entorno de trabajo	2
1. 💡 ¿Qué es este curso y qué haremos?	2
2. 🛠️ Tecnologías que vamos a trabajar	2
3. 💻 Entorno de trabajo y requisitos	3
4. 🏗️ Arquitectura de los laboratorios	4
5. 📖 Estructura de los temas	4
6. 🎓 Evaluación: ¿Cómo superar el curso?	4
7. 🐧 Preparación del entorno anfitrión (Guía para Ubuntu/Debian)	5
7.1 Instalación de VirtualBox	5
7.2 Instalación de Vagrant	5
7.3 Instalación de Terraform	6
7.4 Instalación de Docker Engine	6
8. 📀 Importación de la Máquina Virtual (OVA)	7
8.1 Importación en Oracle VM VirtualBox (Recomendado)	7
8.2 Importación en VMware (Workstation o Player)	7
8.3 El paso final: Habilitar virtualización anidada	8
8.4 Credenciales	8
9. 🧠 Metodología de trabajo y el "Modelo Mental"	8
10. 🌐 Gestión de la Red Bridge (LabCEFIRE)	8
11. 📋 Checklist de "Primeros Auxilios" (Troubleshooting)	9
🔍 1. ¿Está activada la virtualización anidada?	9
🔍 2. ¿Me he quedado sin memoria RAM?	10
🔍 3. ¿He limpiado el laboratorio anterior?	10
🔍 4. Otros problemas frecuentes	11

UNIDAD 01. INTRODUCCIÓN AL CURSO Y ENTORNO DE TRABAJO

1. ¿QUÉ ES ESTE CURSO Y QUÉ HAREMOS?

Bienvenido/a al curso **DevOps. Automatización y monitorización de infraestructuras**. En el ecosistema tecnológico actual, la frontera entre el desarrollo de software y la administración de sistemas ha desaparecido. Ya no es viable configurar servidores a mano ni realizar despliegues manuales.

En este curso daremos el salto hacia la ingeniería de confiabilidad y la automatización total. A lo largo de los próximos temas, no vas a leer simples tutoriales teóricos; vas a construir infraestructuras completas, desde la virtualización de redes y despliegue masivo de sistemas operativos, hasta la orquestación de contenedores, gestión de la configuración, integración continua (CI/CD) y monitorización proactiva.

El objetivo es que adquieras el "modelo mental" de un Arquitecto DevOps: tratar la infraestructura como código (IaC), asumir que los sistemas fallan y prepararlos para que se recuperen automáticamente.

2. TECNOLOGÍAS QUE VAMOS A TRABAJAR

A lo largo del curso manejaremos el stack de herramientas estándar de la industria IT. Los temas del 2 al 6 están diseñados para dominar progresivamente estas tecnologías:

- **Unidad 02. Introducción a la automatización con Vagrant:** Dejaremos atrás las engorrosas instalaciones manuales de sistemas operativos. Aprenderemos los fundamentos de la Infraestructura como Código (IaC) utilizando Vagrant y VirtualBox para crear entornos inmutables y aislados. Veremos cómo definir recursos de hardware en un archivo Vagrantfile y cómo aprovisionar servidores (instalar software y configurar redes) de forma totalmente automatizada mediante scripts modulares, garantizando que el entorno de trabajo sea idéntico cada vez que lo construyamos.
- **Unidad 03. Despliegue automatizado con FOG sobre PXE:** Daremos el salto a la administración útil para sistemas físicos y la clonación masiva simulando un entorno físico corporativo, como un aula de informática o un centro de datos. Configuraremos un servidor FOG, prepararemos una máquina maestra o "Golden Image" (limpiándola con sysprep), y aprenderemos a capturar y desplegar su disco duro a otros equipos vacíos a través de la red local utilizando el arranque PXE, todo gobernado desde un panel centralizado.
- **Unidad 04. Gestión de la configuración con Salt y observabilidad:** Evolucionaremos hacia arquitecturas más ágiles orquestando clústeres de contenedores Docker con Terraform. Para evitar la "deriva de configuración", introduciremos SaltStack, gobernando múltiples nodos de forma centralizada mediante su arquitectura Master-Minion y su bus de alta velocidad. Además, montaremos nuestro primer "cerebro" analítico: un stack de observabilidad total impulsado por Prometheus (para recolectar métricas con Node Exporters) y Grafana (para crear videowalls interactivos de

monitorización).

- **Unidad 05. Automatización y monitorización en Odoon:** Aplicaremos las tecnologías anteriores a un escenario de producción empresarial crítico. Desplegaremos una arquitectura multicontenedor compleja para el ERP Odoon y sus bases de datos PostgreSQL. Llevaremos la monitorización al siguiente nivel diferenciando entre métricas de hardware y de negocio (cuántos usuarios hay activos, ventas registradas), centralizaremos los logs con Alloy y Loki sin usar agentes pesados, y aplicaremos "Ingeniería del Caos" inyectando fallos intencionados para poner a prueba la resiliencia de nuestro sistema.
- **Unidad 06. Automatización y CI/CD Despliegues y Monitorización con Jenkins:** Construiremos una auténtica Factoría Digital de Software para eliminar la intervención manual en las subidas a producción. Implementaremos un pipeline declarativo de Integración y Despliegue Continuo usando Jenkins y un repositorio Git local (Gitea). Observaremos cómo el código fuente viaja de forma autónoma pasando por pruebas de calidad, construcción de imágenes Docker, validación en entornos seguros de Staging y, finalmente, atravesando barreras de aprobación manuales antes de llegar a los usuarios finales.

3. ENTORNO DE TRABAJO Y REQUISITOS

Para llevar a cabo todas las prácticas, necesitas un sistema anfitrión capaz de virtualizar y ejecutar contenedores. Tienes dos opciones para preparar tu entorno de trabajo:

Opción A: Tu propia máquina Linux (Recomendada) Puedes utilizar tu propio equipo si cuentas con una distribución Linux instalada (como Ubuntu o Q4OS). Deberás asegurarte de tener instaladas y configuradas las siguientes herramientas base en tu host:

- VirtualBox (v7.0+)
- Vagrant (v2.3+)
- Docker Engine
- Terraform CLI

Opción B: Máquina Virtual Preconfigurada (Todo en uno) Para facilitar el inicio rápido, hemos preparado una máquina virtual "llave en mano" que ya contiene todo el software y dependencias instaladas.

- **Descarga:** Puedes descargar la imagen OVA desde aquí:
<https://ficticio.cefire.edu.es/descargas/q4os-devops-env.ova>
- **Sistema Operativo:** Q4OS (ligero y basado en Debian).
- **Requisitos de la VM:** Deberás asignarle en tu hipervisor **8GB de memoria RAM** para que pueda ejecutar las herramientas sin asfixiarse.
- **Requisito Crítico:** Debes habilitar obligatoriamente la opción de **Virtualización Anidada (Nested VT-x/AMD-V)** en los ajustes del procesador de tu hipervisor, ya que ejecutaremos máquinas de VirtualBox *dentro* de esta máquina virtual.

4. ARQUITECTURA DE LOS LABORATORIOS

Una de las grandes ventajas de este curso es la flexibilidad de su topología.

Diseño Independiente: Cada laboratorio de las Unidades 02 a 06 está diseñado para ser **totalmente independiente**. Al terminar una práctica, se incluyen scripts de limpieza para demoler la infraestructura y liberar recursos. Esto garantiza que cualquier persona con un PC estándar pueda realizar el curso sin quedarse sin memoria RAM.

Integración y Escalabilidad: Si tienes un equipo potente (por ejemplo, con 16GB o 32GB de RAM), puedes optar por dejar los laboratorios encendidos simultáneamente. La arquitectura de red que crearemos (el bridge lógico **LabCEFIRE**) está pensada para que las máquinas de la Unidad 03 puedan comunicarse e integrarse perfectamente con el panel de monitorización de la Unidad 04[cite: 5]. ¡El límite lo pone tu hardware!

5. ESTRUCTURA DE LOS TEMAS

Para mantener un orden metodológico claro, cada unidad del curso (del tema 2 al 6) está estructurada exactamente de la misma manera. Encontrarás los siguientes recursos:

1. **Teoría del tema:** Un documento PDF detallando los conceptos fundamentales, arquitecturas y buenas prácticas.
2. **Laboratorio práctico:** La guía paso a paso ("Caso Práctico") que te guiará en el despliegue de la infraestructura desde cero.
3. **Ficheros del laboratorio:** Una carpeta con todos los scripts de bash, archivos Vagrantfile o terraform.tf y configuraciones necesarias para que no tengas que escribir todo el código desde la nada.
4. **Cheatsheet:** Una chuleta rápida con los comandos más importantes que usaremos en ese módulo.
5. **Documento de entrega:** Las instrucciones claras sobre qué capturas de pantalla o evidencias debes subir a la plataforma para validar que has completado la práctica.

6. EVALUACIÓN: ¿CÓMO SUPERAR EL CURSO?

Queremos que este curso sea práctico, útil y que se adapte a tus intereses o a la realidad de tu centro/empresa. Por ello, **no es obligatorio realizar los 5 laboratorios para aprobar.**

Para superar el curso y obtener la certificación, **solo necesitas completar satisfactoriamente 3 de los 5 laboratorios.** Puedes elegirlos en cualquier orden. Si te interesa más el despliegue físico, quizás prefieras hacer Vagrant y FOG; si prefieres la nube y los contenedores, puedes saltar directamente a Terraform, Odoo y Jenkins. Tú diseñas tu propio camino de aprendizaje.

No dudes en utilizar los foros del curso para preguntar dudas, compartir errores o debatir sobre arquitecturas. En DevOps, el fallo es parte del aprendizaje. ¡Comencemos!

7. PREPARACIÓN DEL ENTORNO ANFITRIÓN (GUÍA PARA UBUNTU/DEBIAN)

Si has elegido la **Opción A** del punto 3 y prefieres montar el laboratorio en tu propio ordenador utilizando Debian (o distribuciones derivadas como Ubuntu, Linux Mint o Q4OS), a continuación te detallamos los comandos exactos para instalar las últimas versiones de todo el software necesario.

Antes de empezar, abre tu terminal y asegúrate de tener el sistema actualizado y las herramientas básicas de red instaladas:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget gnupg lsb-release apt-transport-https
software-properties-common
```

7.1 Instalación de VirtualBox

VirtualBox actuará como nuestro hipervisor de Tipo 2 principal para los primeros temas del curso. En Debian, podemos instalarlo directamente desde los repositorios oficiales, aunque siempre es recomendable tener la *Extension Pack* para soporte avanzado de red y USB.

```
# 1. Instalar VirtualBox y las dependencias del kernel
sudo apt install -y virtualbox virtualbox-ext-pack linux-headers-$(uname -r)
```

(**Nota:** Durante la instalación del virtualbox-ext-pack, te aparecerá una pantalla azul en la terminal pidiendo que aceptes la licencia de Oracle. Selecciona "Sí").

7.2 Instalación de Vagrant

No utilizaremos la versión de Vagrant que viene por defecto en los repositorios de Debian, ya que suele estar obsoleta. Vamos a añadir el repositorio oficial de HashiCorp para instalar la última versión:

```
# 1. Descargar la clave GPG oficial de HashiCorp
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg

# 2. Añadir el repositorio oficial a nuestro sistema
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list

# 3. Actualizar la lista e instalar Vagrant
sudo apt update
sudo apt install -y vagrant
```

7.3 Instalación de Terraform

Dado que Terraform también es una herramienta desarrollada por HashiCorp, comparte el mismo repositorio que acabamos de configurar en el paso anterior para Vagrant. Por lo tanto, su instalación ahora es directa:

```
# Instalar Terraform directamente (el repositorio ya fue añadido en el paso 7.2)
sudo apt install -y terraform
```

7.4 Instalación de Docker Engine

Para los temas avanzados de orquestación y observabilidad, necesitaremos Docker. Al igual que con Vagrant, instalaremos la versión oficial (Docker CE) en lugar de la versión de los repositorios de Debian.

```
# 1. Crear el directorio para las claves (si no existe)
sudo install -m 0755 -d /etc/apt/keyrings

# 2. Descargar la clave GPG oficial de Docker
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# 3. Configurar el repositorio estable de Docker
echo \
  "deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian \
  "${. /etc/os-release && echo "$VERSION_CODENAME")" stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 4. Instalar Docker Engine y sus plugins
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

 **Paso Crítico Post-Instalación (Permisos de Docker):** Por defecto, Docker requiere privilegios de root (sudo) para ejecutarse. Para que las herramientas como Terraform puedan orquestar contenedores correctamente con tu usuario normal, debes añadirte al grupo docker:

```
sudo usermod -aG docker $USER
```

Para que este último cambio de permisos surta efecto, **debes cerrar la sesión de tu usuario y volver a entrar**, o bien reiniciar el ordenador.

Una vez completados estos cuatro pasos, tu máquina estará perfectamente calibrada para afrontar con éxito cualquier laboratorio de este curso.

Aquí tienes el punto 8, diseñado para guiar al alumnado en la importación de la máquina virtual preconfigurada, cubriendo los dos hipervisores más comunes:

8. IMPORTACIÓN DE LA MÁQUINA VIRTUAL (OVA)

Si has optado por utilizar la máquina virtual proporcionada por el curso (Opción B), el archivo que has descargado tiene una extensión .ova (Open Virtualization Format). Este es un estándar de la industria que permite empaquetar el disco duro, la configuración de hardware y el sistema operativo en un solo fichero listo para ser importado.

8.1 Importación en Oracle VM VirtualBox (Recomendado)

VirtualBox es la plataforma principal sobre la que se han diseñado los laboratorios, por lo que la compatibilidad es total.

1. Abrir el asistente: Ve al menú Archivo > Importar servicio virtual... (o pulsa Ctrl + I).
2. Seleccionar el archivo: Busca el fichero .ova que descargaste previamente.
3. Configuración de importación: Se abrirá una ventana con los detalles de la máquina.
 - Nombre: Puedes dejar el nombre por defecto o cambiarlo a Entorno DevOps - Tema 1.
 - RAM: Asegúrate de que marca 8192 MB (8GB). Si tu PC tiene poca RAM, podrías intentar bajarlo a 6GB, pero el rendimiento de los laboratorios avanzados se verá afectado.
 - Política de dirección MAC: Selecciona "Generar nuevas direcciones MAC para todas las tarjetas de red". Esto evitará conflictos si otros compañeros usan la misma red.
4. Finalizar: Pulsa en Importar. El proceso tardará unos minutos dependiendo de la velocidad de tu disco duro.

8.2 Importación en VMware (Workstation o Player)

Si prefieres usar el ecosistema de VMware, también puedes importar el archivo OVA, aunque el proceso realiza una conversión interna.

1. Abrir el archivo: Ve a File > Open y selecciona el archivo .ova.
2. Almacenamiento: VMware te pedirá una ruta en tu disco duro para guardar la nueva máquina virtual convertida al formato .vmx.
3. Conversión: Pulsa en Import. Es posible que aparezca un aviso sobre "falla de conformidad OVF" (OVF specification conformance); si esto ocurre, pulsa en Retry o Relax para forzar la importación.
4. Ajustes finales: Una vez importada, entra en Edit virtual machine settings y verifica que la memoria RAM está en 8GB.

8.3 El paso final: Habilitar virtualización anidada

Este paso es el más importante. Dado que dentro de esta máquina virtual vamos a lanzar otras máquinas virtuales (especialmente en la Unidad 02 y 03), necesitamos activar el "paso" de la tecnología de virtualización del procesador físico a la VM.

- **En VirtualBox:** Con la máquina apagada, ve a Configuración > Sistema > Procesador. Marca la casilla Habilitar VT-x/AMD-V anidado.
 - **Nota:** Si la casilla aparece en gris y no puedes marcarla, abre una terminal en tu host y ejecuta: `VBoxManage modifyvm "Nombre_de_tu_VM" --nested-hw-virt on`
- En VMware: Ve a Processors y marca la casilla Virtualize Intel VT-x/EPT or AMD-V/RVI.

8.4 Credenciales

Ya puedes arrancar la máquina. Las credenciales de acceso por defecto serán:

- **Usuario (con permisos de sudo):** alumno
- **Contraseña:** alumno

9. METODOLOGÍA DE TRABAJO Y EL "MODELO MENTAL"

Para no perder la cordura entre tantos scripts y máquinas virtuales, el alumno debe entender la jerarquía de ejecución que usaremos siempre:

- **Scripts de Orquestación (Host):** Se ejecutan en tu máquina física (o en la VM principal) para crear redes o llamar a Terraform/Vagrant.
- **Scripts de Aprovisionamiento (Guest):** Son el "ADN" que se inyecta dentro de las máquinas virtuales o contenedores para instalar el software.
- **Idempotencia:** Es un concepto vital en este curso. Los scripts están diseñados para poder ejecutarse varias veces sin romper nada; si algo ya está instalado, el script simplemente lo saltará.

10. GESTIÓN DE LA RED BRIDGE (LabCEFIRE)

Este es el punto donde suelen fallar los laboratorios. Es fundamental explicar que:

- Casi todos los laboratorios crean un "conmutador virtual" llamado **LabCEFIRE** en el kernel de tu Linux.
- Esta red utiliza el segmento **172.28.0.0/24** y cada laboratorio usa IPs según su número (por ejemplo la unidad 02 utiliza 20, 21,... etc. o la unidad 03, utiliza 30, 31, ..., etc.).
- **Crítico:** Si el alumno tiene otra red (VPN, Docker interno u otro laboratorio) usando ese mismo rango, habrá un conflicto. Se recomienda apagar VPNs corporativas antes de lanzar los scripts de red.

11. CHECKLIST DE "PRIMEROS AUXILIOS" (TROUBLESHOOTING)

Antes de preguntar dudas en el foro, el alumno debe verificar estos tres puntos que aparecen como "problemas comunes" en todas las unidades:

1. ¿Está activada la virtualización anidada?

Síntoma en la fábrica: Al intentar arrancar las máquinas pesadas de las Unidades 02 y 03 (las controladas por Vagrant), el proceso se detiene en seco con un error críptico: VT-x is not available (VERR_VMX_NO_VMX). Es como si el motor de arranque girase pero no hubiera chispa en las bujías.

Causa probable: El hipervisor (VirtualBox, VMWare) no tiene permiso para usar las instrucciones de virtualización del procesador. Esto ocurre en tres niveles:

- BIOS/UEFI del host físico: La virtualización (Intel VT-x, AMD-V) puede estar deshabilitada en la placa base.
- Hipervisor anfitrión: VirtualBox no tiene activada la opción de "virtualización anidada" para la máquina virtual concreta.
- Windows como host: El sistema operativo anfitrión puede tener Hyper-V, el Subsistema de Windows para Linux (WSL) o el "Aislamiento del núcleo" acaparando las extensiones de virtualización, impidiendo que VirtualBox las use.

Solución (comprobaciones en orden):

1. En la BIOS/UEFI del PC físico: Reinicia y entra en la configuración de la placa base. Busca y activa:
 - Intel Virtualization Technology (VT-x) o AMD-V
 - En algunas placas (Gigabyte y otras) también es necesario activar VT-d (virtualización de E/S).
2. En VirtualBox (si usas la OVA del curso): Ejecuta en el terminal del host:
text
VBoxManage modifyvm "NombreDeLaMV" --nested-hw-virt on
Sustituye "NombreDeLaMV" por el nombre exacto de tu máquina virtual.
3. En Windows (si es tu sistema anfitrión):
 - Desactiva Hyper-V y características relacionadas desde PowerShell (como Administrador):
text
bcdedit /set hypervisorlaunchtype off
 - Ve a Panel de Control > Programas > Activar o desactivar características de Windows y desmarca:
 - Hyper-V
 - Plataforma de máquina virtual
 - Plataforma de hipervisor de Windows
 - Subsistema de Windows para Linux (si no lo necesitas)
 - Desactiva el "Aislamiento del núcleo": Configuración > Seguridad de Windows >

Seguridad del dispositivo > Aislamiento del núcleo > Desactivar "Integridad de memoria".

- Importante: Después de estos cambios, apaga y reinicia completamente el equipo (no vale reiniciar, debe ser un apagado completo para que el hipervisor se libere).

Nota del arquitecto: Si después de todo esto el problema persiste, la solución más limpia es instalar Vagrant directamente en el host físico y lanzar allí los laboratorios 02 y 03. La OVA del curso es solo una comodidad; Vagrant se instala con un simple instalador y se desinstala igual de fácil cuando ya no se necesite.

2. ¿Me he quedado sin memoria RAM?

Síntoma en la fábrica: Contenedores o máquinas virtuales que desaparecen sin dejar rastro. No hay un error claro en los logs, simplemente el proceso ya no está. Es como si un operario hubiera sido despedido a mitad de turno sin avisar.

Causa probable: El PC anfitrión se ha quedado sin memoria RAM disponible. Cuando esto ocurre, el kernel de Linux invoca al implacable OOM Killer (Out-Of-Memory Killer), que selecciona uno o varios procesos y los termina fulminantemente para liberar memoria. Con frecuencia, los elegidos son los contenedores Docker o las máquinas virtuales, que caen sin previo aviso en mitad de un despliegue crítico.

Solución:

- Antes de lanzar un laboratorio nuevo: Ejecuta los scripts de limpieza del laboratorio anterior (suelen ser 98-destroy-all.sh o 99-destroy-network.sh). Esto libera RAM, CPU y evita colisiones de puertos e IPs.
- Monitoriza la RAM libre: En Linux, free -h; en Windows, el Administrador de Tareas.
- Reduce la asignación de RAM: Si usas la OVA del curso, asigna solo la RAM necesaria (8 GB suele ser suficiente, pero si tu equipo tiene 16 GB totales, no le des 12 a la máquina virtual).
- Cierra aplicaciones innecesarias en el host antes de lanzar los laboratorios (navegadores con muchas pestañas, editores pesados, etc.).

3. ¿He limpiado el laboratorio anterior?

Síntoma en la fábrica: Al arrancar un nuevo laboratorio aparecen errores de "puerto ya en uso", "IP ya asignada" o "el recurso ya existe". Es como si intentaras construir una nave nueva sobre los cimientos de la anterior sin haberla demolido.

Causa probable: Los laboratorios de este curso usan redes privadas con direcciones IP fijas (subred 172.28.0.0/24) y puertos estándar (3000, 8080, 9090, etc.). Si los contenedores o máquinas virtuales del laboratorio anterior siguen ejecutándose, los nuevos no podrán ocupar esos recursos.

Solución:

- Antes de empezar un tema nuevo: Ejecuta el script de destrucción del laboratorio anterior. Generalmente se llaman 98-destroy-infra.sh o 99-destroy-network.sh y se encuentran en la carpeta scripts-laboratorio/ de cada unidad.

Si no recuerdas qué laboratorio tenías activo: Haz una limpieza general:

bash

```
docker stop $(docker ps -aq) 2>/dev/null
```

```
docker rm $(docker ps -aq) 2>/dev/null
```

- `docker network prune -f`
- Para VirtualBox: Revisa en la interfaz gráfica que no haya máquinas virtuales en ejecución de laboratorios anteriores.
- Regla de oro: Si tu equipo tiene recursos limitados, la limpieza entre laboratorios no es una recomendación, es una obligación. Solo así evitarás colisiones de puertos e IPs y el OOM Killer no aparecerá por sorpresa.



4. Otros problemas frecuentes

- Timeout al arrancar máquinas virtuales (Vagrant): A veces el primer vagrant up tarda tanto que el script de despliegue supera el tiempo de espera. Simplemente vuelve a lanzar el script una segunda vez; la máquina ya estará parcialmente iniciada y arrancará más rápido.
- Conflicto entre hipervisores (Windows): No puedes tener Docker, VirtualBox y VMWare ejecutándose simultáneamente. Todos compiten por las mismas instrucciones de virtualización del procesador. Apaga los que no estés usando.
- Error hv.capable en VMWare: Indica que Hyper-V está activo en Windows y bloquea a VMWare. La solución es la misma que para VirtualBox: desactiva Hyper-V con `bcdedit /set hypervisorlaunchtype off` y reinicia completamente el equipo.